



ĐẠI HỌC QUỐC GIA HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO
Link and Network with SDN
MÔN: MẠNG MÁY TÍNH NÂNG CAO

Sinh viên thực hiện:

1. Nguyễn Phan Hùng Linh - MSSV: 23127081
2. Lương Văn Dũng - MSSV: 23127353

MỤC LỤC

I. MININET PRIMER (TOPOLOGY).....	3
1.1. Mục tiêu.....	3
1.2. Triển khai (Implementation).....	3
1.3. Kết quả thực nghiệm.....	3
1.3.1. Kiểm tra cấu trúc mạng (Lệnh dump).....	3
1.3.2. Kiểm tra kết nối (Lệnh pingall).....	4
1.3.3. Đo băng thông (Lệnh iperf).....	5
II. LEARNING SWITCH & BASIC FIREWALL.....	5
2.1. Giải pháp (Implementation).....	5
2.2. Kết quả (Deliverables).....	6
III. COMPLEX NETWORK (STATIC ROUTING & FIREWALL).....	7
3.1. Mục tiêu.....	7
3.2. Chiến lược thiết kế.....	7
Chiến lược thiết kế dựa trên cơ chế phân tầng độ ưu tiên trong OpenFlow, cụ thể như sau:.....	7
a. Tầng 1: An ninh và Bảo mật - Priority 200.....	7
b. Tầng 2: Nền tảng kết nối - Priority 100.....	8
c. Tầng 3: Định tuyến tĩnh - Priority 10.....	8
3.3. Kết quả thực nghiệm.....	8
3.3.1. Thiết lập môi trường thực nghiệm.....	8
3.3.2. Kiểm tra tính thông suốt và chặn ICMP (Lệnh pingall).....	9
3.3.3. Kiểm tra tường lửa lớp ứng dụng (Lệnh iperf).....	9
3.3.4. Kiểm chứng băng luồng.....	10
IV. DYNAMIC HANDLING / ADVANCED FEATURES.....	11
4.1. Giải pháp (Implementation).....	11
4.2. Kết quả (Deliverables).....	15
V. TÀI LIỆU THAM KHẢO.....	12

I. MININET PRIMER (TOPOLOGY)

1.1. Mục tiêu

- Tạo lập topo mạng hình sao (Star Topology) sử dụng Python script trong Mininet..
- Làm quen với các công cụ gỡ lỗi cơ bản: `dump`, `pingall`, `iperf`.

1.2. Triển khai (Implementation)

a. Thiết kế Mô hình mạng (Topology Design): Xây dựng cấu trúc mạng hình sao (Star Topology) thông qua lớp đối tượng kế thừa từ thư viện `Topo` của Mininet. Cấu trúc cụ thể như sau:

- Nút trung tâm: Một thiết bị chuyển mạch ảo (Switch) được đặt tên định danh là `s1`.

- Nút biên: Bốn máy trạm (Host) được đặt tên lần lượt là **h1**, **h2**, **h3**, **h4**.
- Liên kết: Mỗi host được nối trực tiếp vào switch **s1**.

b. Cấu hình vận hành Switch Để đảm bảo mạng hoạt động được ngay lập tức mà chưa cần bộ điều khiển (Controller) phức tạp, thiết lập chế độ vận hành cho Switch **s1** là Standalone Mode.

- *Cơ chế*: Trong chế độ này, Switch sẽ hoạt động như một thiết bị Lớp 2 truyền thống (Legacy Switch). Nó tự động xây dựng bảng địa chỉ MAC (MAC Table) thông qua quá trình học địa chỉ nguồn (Source MAC Learning) từ các gói tin đến.
- *Mục đích*: Việc này khắc phục vấn đề mặc định của Open vSwitch (thường drop gói tin khi mất kết nối với Controller), đảm bảo lệnh **pingall** thành công ngay lập tức.

1.3. Kết quả thực nghiệm

Để kiểm chứng tính đúng đắn của topo mạng và hiệu năng kết nối. Thực hiện các lệnh kiểm tra tiêu chuẩn trên giao diện dòng lệnh (CLI) của Mininet. Dưới đây là phân tích chi tiết dựa trên kết quả thu được:

1.3.1. Kiểm tra cấu trúc mạng (Lệnh dump)

```
mininet@mininet-vm:~/461_mininet/topos$ sudo python part1.py
--- Dumping host connections ---
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
--- Testing network connectivity ---
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=7565>
<Host h2: h2-eth0:10.0.0.2 pid=7567>
<Host h3: h3-eth0:10.0.0.3 pid=7569>
<Host h4: h4-eth0:10.0.0.4 pid=7585>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None,s1-eth4:None pid=7590>
mininet> []
```

Hình 1.3.1: Kết quả hiển thị các node và liên kết mạng.

Nhận xét:

- Kết quả từ lệnh **dump** xác nhận cấu trúc mạng hình sao đã được khởi tạo chính xác theo mã nguồn **part1.py**.
- Các Host: 4 máy trạm (**h1**, **h2**, **h3**, **h4**) đã được gán địa chỉ IP lần lượt từ **10.0.0.1** đến **10.0.0.4**. Mỗi host có một tiến trình (PID) riêng biệt (ví dụ: h1 có PID 7565), chứng tỏ chúng đang hoạt động độc lập.
- Liên kết: Switch **s1** (PID 7590) đã thiết lập 4 cổng kết nối tương ứng với 4 host:
 - **h1-eth0** nối vào **s1-eth1**
 - **h2-eth0** nối vào **s1-eth2**
 - **h3-eth0** nối vào **s1-eth3**
 - **h4-eth0** nối vào **s1-eth4**
- Điều này chứng minh cấu trúc topo mạng đã được định nghĩa đúng.

1.3.2. Kiểm tra kết nối (Lệnh `pingall`)

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet> █
```

Hình 1.3.2: Kết quả kiểm tra tính thông suốt giữa các host.

Nhận xét:

- Lệnh `pingall` gửi các gói tin ICMP Echo Request giữa tất cả các cặp máy trạm.
- Kết quả hiển thị: **Results: 0% dropped (12/12 received)**.
- Phân tích: Tỷ lệ mất gói tin là 0% chứng tỏ Switch `s1` (đang chạy ở chế độ Standalone) đã thực hiện chức năng học địa chỉ MAC và chuyển mạch hoàn hảo. Mọi host đều có thể giao tiếp với nhau mà không gặp trở ngại nào.

1.3.3. Đo băng thông (Lệnh `iperf`)

```
mininet> iperf h1 h2
*** Iperf: testing TCP bandwidth between h1 and h2
*** Results: ['32.0 Gbits/sec', '32.0 Gbits/sec']
mininet> iperf h1 h3
*** Iperf: testing TCP bandwidth between h1 and h3
*** Results: ['31.0 Gbits/sec', '31.1 Gbits/sec']
mininet> iperf h1 h4
*** Iperf: testing TCP bandwidth between h1 and h4
*** Results: ['25.5 Gbits/sec', '25.5 Gbits/sec']
mininet> iperf h2 h3
*** Iperf: testing TCP bandwidth between h2 and h3
*** Results: ['31.2 Gbits/sec', '31.3 Gbits/sec']
mininet> iperf h2 h4
*** Iperf: testing TCP bandwidth between h2 and h4
*** Results: ['31.5 Gbits/sec', '31.6 Gbits/sec']
mininet> iperf h3 h4
*** Iperf: testing TCP bandwidth between h3 and h4
*** Results: ['31.9 Gbits/sec', '31.9 Gbits/sec']
mininet> █
```

Hình 1.3.3: Kết quả đo băng thông TCP giữa các cặp host.

Nhận xét:

- Số liệu: Bảng thông ghi nhận được rất cao và ổn định, dao động trong khoảng từ 25.5 Gbits/sec đến 32.0 Gbits/sec.
 - Cao nhất: Cặp h1-h2 đạt 32.0 Gbits/sec.
 - Thấp nhất: Cặp h1-h4 đạt 25.5 Gbits/sec.
- Kết luận: Tốc độ này phản ánh đặc thù của môi trường mạng ảo hóa Mininet, nơi các liên kết là ảo và tốc độ truyền tải phụ thuộc vào tốc độ xử lý của CPU/RAM máy chủ thay vì bị giới hạn bởi dây cáp vật lý. Kết nối đảm bảo đủ hiệu năng cho các ứng dụng mạng yêu cầu băng thông lớn.

II. LEARNING SWITCH & BASIC FIREWALL

2.1. Giải pháp (Implementation)

Có 3 rule với 3 priority lần lượt là

- Priority 100: Match các packet có dl_type = 0x0806 (là ARP packet), nếu đúng thì thêm 1 action kêu switch flood packet ra mọi cổng trừ cổng vào
- Priority 90: Match các packet có dl_type = 0x0800 (là IP packet) và nw_proto = 1 (là ICMP packet), nếu đúng thì thêm 1 action kêu switch flood packet ra mọi cổng trừ cổng vào
- Priority 10: Match các packet có dl_type = 0x0800 (là IP packet), nếu đúng thì bỏ trống action - tương đương kêu switch không làm gì cả. Switch sẽ tự động drop packet đó.

2.2. Kết quả (Deliverables)

```
root@huli-VMware20-1:/home/huli/pox# ./pox.py openflow.of_01 --port=6653 misc.part2c
controller
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:Support for Python 3 is experimental.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
Unhandled packet :[00:00:00:00:00:04>33:33:ff:00:00:04 IPV6][IPv6 ::>ff02::1:ff00:4
ICMP6][ICMP+TYPE_NEIGHBOR_SOLICITATION/0][ICMPv6/NDNeighborSolicitation num_opts:1 o
pts:[<NDP Option Type 14>] target:fe80::200:ff:fe00:4]
Unhandled packet :[00:00:00:00:00:01>33:33:ff:00:00:01 IPV6][IPv6 ::>ff02::1:ff00:1
ICMP6][ICMP+TYPE_NEIGHBOR_SOLICITATION/0][ICMPv6/NDNeighborSolicitation num_opts:1 o
pts:[<NDP Option Type 14>] target:fe80::200:ff:fe00:1]
Unhandled packet :[00:00:00:00:00:02>33:33:00:00:00:16 IPV6][IPv6 ::>ff02::16 HOP_OP
TS+ICMP6][ICMP+TYPE_MC_LISTENER_REPORT_V2/0][44 bytes: 00 00 00 02 03 ...]
Unhandled packet :[00:00:00:00:00:02>33:33:ff:00:00:02 IPV6][IPv6 ::>ff02::1:ff00:2
ICMP6][ICMP+TYPE_NEIGHBOR_SOLICITATION/0][ICMPv6/NDNeighborSolicitation num_opts:1 o
pts:[<NDP Option Type 14>] target:fe80::200:ff:fe00:2]
Unhandled packet :[00:00:00:00:00:04>33:33:00:00:00:16 IPV6][IPv6 ::>ff02::16 HOP_OP
TS+ICMP6][ICMP+TYPE_MC_LISTENER_REPORT_V2/0][44 bytes: 00 00 00 02 03 ...]
Unhandled packet :[00:00:00:00:00:01>33:33:00:00:00:16 IPV6][IPv6 ::>ff02::16 HOP_OP
TS+ICMP6][ICMP+TYPE_MC_LISTENER_REPORT_V2/0][44 bytes: 00 00 00 02 03 ...]
Unhandled packet :[00:00:00:00:00:03>33:33:00:00:00:16 IPV6][IPv6 ::>ff02::16 HOP_OP
TS+ICMP6][ICMP+TYPE_MC_LISTENER_REPORT_V2/0][44 bytes: 00 00 00 02 03 ...]
Unhandled packet :[00:00:00:00:00:03>33:33:ff:00:00:03 IPV6][IPv6 ::>ff02::1:ff00:3
ICMP6][ICMP+TYPE_NEIGHBOR_SOLICITATION/0][ICMPv6/NDNeighborSolicitation num_opts:1 o
pts:[<NDP Option Type 14>] target:fe80::200:ff:fe00:3]
Unhandled packet :[00:00:00:00:00:04>33:33:00:00:00:16 IPV6][IPv6 fe80::200:ff:fe00:
```

```

root@huli-VMware20-1:/home/huli# sudo python3 part2.py
mininet> dump
<Host h1: h1-eth0:10.0.1.2 pid=106413>
<Host h2: h2-eth0:10.0.0.2 pid=106415>
<Host h3: h3-eth0:10.0.0.3 pid=106417>
<Host h4: h4-eth0:10.0.1.3 pid=106419>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None,s1-eth4:None pid=
106424>
<RemoteController c0: 127.0.0.1:6653 pid=106407>
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X h4
h2 -> X h3 X
h3 -> X h2 X
h4 -> h1 X X
*** Results: 66% dropped (4/12 received)
mininet> iperf
*** Iperf: testing TCP bandwidth between h1 and h4
^C
Interrupt
mininet> dpctl dump-flows
*** s1 -----
cookie=0x0, duration=202.398s, table=0, n_packets=10, n_bytes=420, priority=100,arp
actions=FLOOD
cookie=0x0, duration=202.359s, table=0, n_packets=9, n_bytes=666, priority=10,ip ac
tions=drop
cookie=0x0, duration=202.359s, table=0, n_packets=8, n_bytes=784, priority=90,icmp
actions=FLOOD
mininet>

```

III. COMPLEX NETWORK (STATIC ROUTING & FIREWALL)

3.1. Mục tiêu

- Xây dựng Controller bằng POX để quản lý định tuyến cho mạng doanh nghiệp giả lập.
- Thực hiện định tuyến tĩnh (Static Routing) dựa trên IP đích.
- Cấu hình Tường lửa (Firewall) để ngăn chặn truy cập trái phép từ Internet (**hnotrust**).

3.2. Chiến lược thiết kế

Chiến lược thiết kế dựa trên cơ chế phân tầng độ ưu tiên trong OpenFlow, cụ thể như sau:

a. Tầng 1: An ninh và Bảo mật - Priority 200

Đây là lớp phòng thủ đầu tiên với độ ưu tiên cao nhất. Mọi gói tin đi vào Core Switch sẽ được kiểm tra qua màng lọc này trước:

- Chặn truy cập máy chủ: Thiết lập quy tắc cấm mọi giao tiếp IP từ vùng không tin cậy (**hnotrust1**) đến máy chủ dữ liệu (**serv1**). Hành động là loại bỏ gói tin.

- Chặn dò quét: Để ngăn chặn việc phát hiện cấu trúc mạng nội bộ. Thiết lập quy tắc cấm giao thức ICMP (Ping) từ **hnotrust1** đến toàn bộ các host nội bộ.
- Lý do thiết kế: Việc đặt Priority 200 (cao hơn Routing) đảm bảo rằng dù có đường đi hợp lệ trong bảng định tuyến, gói tin vi phạm chính sách bảo mật vẫn sẽ bị tiêu hủy ngay lập tức.

b. Tầng 2: Nền tảng kết nối - Priority 100

Để các thiết bị mạng giao tiếp được với nhau qua IP, chúng cần phân giải địa chỉ MAC thông qua giao thức ARP.

- Cơ chế: Thiết lập quy tắc cho phép quảng bá toàn bộ gói tin ARP (**dl_type=0x0806**).
- Lý do thiết kế: Nếu không có tầng này (hoặc độ ưu tiên thấp hơn tầng chặn), các máy trạm sẽ không thể tìm thấy nhau, dẫn đến tê liệt toàn bộ mạng trước khi kịp truyền dữ liệu.

c. Tầng 3: Định tuyến tĩnh - Priority 10

Đây là tầng xử lý cuối cùng cho các gói tin hợp lệ (đã vượt qua Firewall và đã hoàn tất ARP).

- Cơ chế: Dựa trên cấu trúc liên kết mạng, Controller sẽ ánh xạ địa chỉ IP đích đến cổng vật lý cụ thể trên **cores21**:
 - Đích **10.0.1.0/24** → Chuyển tiếp ra Port 1 (hướng về Switch **s1**).
 - Đích **10.0.2.0/24** → Chuyển tiếp ra Port 2 (hướng về Switch **s2**).
 - Đích **10.0.3.0/24** → Chuyển tiếp ra Port 3 (hướng về Switch **s3**).
 - Đích **10.0.4.0/24** → Chuyển tiếp ra Port 4 (hướng về Switch **dcs31**).
 - Đích **172.16.10.0/24** → Chuyển tiếp ra Port 5 (hướng về **hnotrust1**).

3.3. Kết quả thực nghiệm

Quá trình kiểm thử được thực hiện nhằm xác minh hai mục tiêu chính: Khả năng định tuyến đúng của các máy nội bộ và Khả năng chặn đứng của tường lửa đối với máy **hnotrust1**. Trước khi đi đến phần test thì khởi tạo kết nối giữa topology với controller

3.3.1. Thiết lập môi trường thực nghiệm

Trước khi tiến hành các kịch bản đo đạc, hệ thống cần được khởi tạo theo quy trình nghiêm ngặt để đảm bảo Bộ điều khiển (Controller) thiết lập kết nối ổn định với Switch lõi (**core s21**). Quy trình thực hiện như sau:

Bước 1: Trên cửa sổ terminal thứ nhất, kích hoạt Controller với quyền quản trị để lắng nghe các sự kiện từ mạng:

```
mininet@mininet-vm:~/461_mininet/topos$ sudo ~/pox/pox.py misc.part3controller info.packet_dump samples.pretty_log log.level --DEBUG
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:info.packet_dump:Packet dumper running
[core] POX 0.7.0 (gar) going up...
[core] Running on CPython (3.8.5/Jul 28 2020 12:59:40)
[core] Platform is Linux-5.4.0-42-generic-x86_64-with-glibc2.29
[version] Support for Python 3 is experimental.
[core] POX 0.7.0 (gar) is up.
[openflow.of_01] Listening on 0.0.0.0:6633
```

Hình 3.3.1a: Khởi chạy Controller

Mục đích: Controller cần sẵn sàng trước để nạp ngay các luồng quy tắc (Flow Rules) khi Switch vừa khởi động (Proactive mode).

Bước 2: Triển khai Topology Trên cửa sổ terminal thứ hai

```
mininet@mininet-vm:~/461_mininet/topos$ sudo python part3.py
Unable to contact the remote controller at 127.0.0.1:6653
mininet> █
```

Hình 3.3.1b: Chạy mã nguồn khởi tạo mạng Mininet

3.3.2. Kiểm tra tính thông suốt và chặn ICMP (Lệnh `pingall`)

```
mininet@mininet-vm:~/461_mininet/topos$ sudo python part3.py
Unable to contact the remote controller at 127.0.0.1:6653
mininet> pingall
*** Ping: testing ping reachability
h10 -> h20 h30 X serv1
h20 -> h10 h30 X serv1
h30 -> h10 h20 X serv1
hnotrust1 -> X X X X
serv1 -> h10 h20 h30 X
*** Results: 40% dropped (12/20 received)
```

Hình 3.3.2: Ma trận kết quả lệnh ping all với tỷ lệ drop 40%

Nhận xét: Tường lửa đã cách ly hoàn toàn `hnotrust1` khỏi việc dò quét mạng bằng giao thức ICMP.

3.3.3. Kiểm tra tường lửa lớp ứng dụng (Lệnh `iperf`)

Kịch bản A: Truy cập hợp lệ (`hnotrust1` → `h10`)

```
mininet> iperf hnotrust1 h10
*** Iperf: testing TCP bandwidth between hnotrust1 and h10
*** Results: ['25.1 Gbits/sec', '25.2 Gbits/sec']
```

Hình 3.3.3a: Kết nối TCP thành công tới máy trạm nội bộ

Nhận xét: Tường lửa hoạt động chính xác theo nguyên tắc "Allow IP traffic", không chặn nhằm các lưu lượng truy cập hợp lệ vào mạng doanh nghiệp.

Kịch bản B: Truy cập trái phép (`hnotrust1` → `serv1`)

```
mininet> iperf hnotrust1 serv1
*** Iperf: testing TCP bandwidth between hnotrust1 and serv1
^C
```

Hình 3.3.3b: Kết nối tới Server bị chặn đứng và gây treo tiến trình

Hiện tượng:

- Khi thực hiện lệnh `iperf hnotrust1 serv1`, màn hình terminal rơi vào trạng thái treo và không hiển thị bất kỳ thông số băng thông nào. Phải sử dụng tổ hợp phím `Ctrl + C` để ngắt tiến trình kiểm thử.

Nhận xét: Tường lửa đã bảo vệ tuyệt đối máy chủ `serv1`, ngăn chặn hoàn toàn việc thiết lập phiên kết nối TCP (TCP Handshake) từ nguồn không xác thực.

3.3.4. Kiểm chứng bằng luồng(Lệnh `dpctl dump-flows`).

```
mininet> dpctl dump-flows
*** cores21 -----
cookie=0x0, duration=1169.132s, table=0, n_packets=41, n_bytes=3082, priority=200,ip,nw_src=172.16.10.100,nw_dst=10.0.4.10 actions=drop
cookie=0x0, duration=1169.133s, table=0, n_packets=2, n_bytes=196, priority=200,icmp,nw_src=172.16.10.100,nw_dst=10.0.1.10 actions=drop
cookie=0x0, duration=1169.133s, table=0, n_packets=2, n_bytes=196, priority=200,icmp,nw_src=172.16.10.100,nw_dst=10.0.2.20 actions=drop
cookie=0x0, duration=1169.133s, table=0, n_packets=2, n_bytes=196, priority=200,icmp,nw_src=172.16.10.100,nw_dst=10.0.3.30 actions=drop
cookie=0x0, duration=1169.133s, table=0, n_packets=0, n_bytes=0, priority=200,icmp,nw_src=172.16.10.100,nw_dst=10.0.4.10 actions=drop
cookie=0x0, duration=1169.133s, table=0, n_packets=0, n_bytes=0, priority=100,arp actions=FLOOD
cookie=0x0, duration=1169.134s, table=0, n_packets=341962, n_bytes=15782660988, priority=10,ip,nw_dst=10.0.1.10 actions=output:"cores21-eth1"
cookie=0x0, duration=1169.134s, table=0, n_packets=6, n_bytes=588, priority=10,ip,nw_dst=10.0.2.20 actions=output:"cores21-eth2"
cookie=0x0, duration=1169.134s, table=0, n_packets=6, n_bytes=588, priority=10,ip,nw_dst=10.0.3.30 actions=output:"cores21-eth3"
cookie=0x0, duration=1169.134s, table=0, n_packets=6, n_bytes=588, priority=10,ip,nw_dst=10.0.4.10 actions=output:"cores21-eth4"
cookie=0x0, duration=1169.134s, table=0, n_packets=274493, n_bytes=18116670, priority=10,ip,nw_dst=172.16.10.100 actions=output:"cores21-eth5"
*** dcs31 -----
cookie=0x0, duration=1169.168s, table=0, n_packets=13, n_bytes=1274, actions=FLOOD
*** s1 -----
cookie=0x0, duration=1169.206s, table=0, n_packets=616458, n_bytes=15800783952, actions=FLOOD
*** s2 -----
cookie=0x0, duration=1169.233s, table=0, n_packets=13, n_bytes=1274, actions=FLOOD
*** s3 -----
cookie=0x0, duration=1169.262s, table=0, n_packets=13, n_bytes=1274, actions=FLOOD
mininet>
```

Hình 3.3.4: Trạng thái bảng luồng tại Switch

Phân tích số liệu thực tế: Dựa vào hình ảnh trên, ta thấy rõ sự phân tầng độ ưu tiên đúng như thiết kế:

1. Tầng Firewall (Priority = 200):

- Dòng đầu tiên:
`priority=200,ip,nw_src=172.16.10.100,nw_dst=10.0.4.10 actions=drop.`
 - *Ý nghĩa:* Chặn IP từ `hnotrust1` đến Server.
 - *Chứng cứ:* Số liệu `n_packets=41` xác nhận rằng đã có 41 gói tin tấn công (từ lệnh `iperf` treo trước đó) bị quy tắc này bắt được và tiêu hủy.
- Các dòng tiếp theo (`icmp`): Chặn ICMP tới các host. Các chỉ số `n_packets=2` chứng tỏ các gói tin ping thử nghiệm đã bị chặn lại tại đây.

2. Tầng Nền tảng (Priority = 100):

- Dòng: `priority=100,arp actions=FLOOD.`
- *Ý nghĩa:* Cho phép giao thức ARP hoạt động trên toàn mạng để phân giải địa chỉ MAC.

3. Tầng Định tuyến (Priority = 10):

- Các dòng có `actions=output:"cores21-eth..."`.
- *Ý nghĩa:* Chuyển tiếp gói tin hợp lệ.
- *Chứng cứ:* Chỉ số `n_packets` rất lớn (ví dụ: 341,962 packets) cho thấy lưu lượng truy cập hợp lệ (như lệnh `iperf` giữa `h10` và `h20`) được thông qua và xử lý bởi các quy tắc này.

Kết luận: Switch *cores21* thực thi đúng logic điều khiển từ Controller, bảo đảm an toàn và hiệu năng định tuyến. Các switch biên cũng thực hiện đúng vai trò gom và chuyển tiếp lưu lượng một cách trong suốt, không tạo ra điểm nghẽn cho hệ thống.

IV. DYNAMIC HANDLING / ADVANCED FEATURES

4.1. Giải pháp (Implementation)

Một số biến

1) `self.arpCache`

- Dict lưu thông tin học được từ ARP và IP packet.
- Key là IPAddr của một host.
- Value là tuple gồm (MAC của host, cổng inPort trên cores21 nơi host đó nằm phía sau).

2) `self.pendingPackets`

- Dict dùng để giữ các IP packet đang chờ vì chưa biết MAC của IP đích.
- Key là IPAddr đích.
- Value là danh sách các packetData (raw bytes) cần forward sau khi học được MAC.

3) `self.gatewayIps`

- Dict mô tả IP gateway cho từng cổng của cores21.
- Key là số port (1..5).
- Value là IP gateway tương ứng subnet.

4) `self.gatewayMacs`

- Dict mô tả MAC router interface cho từng port của cores21.
- Key là số port (1..5).
- Value là MAC ảo của gateway.

Chức năng các hàm chính

`init`

- Lưu connection và đăng ký listener để nhận PacketIn.
- Khởi tạo các cấu trúc dữ liệu bên trên:
- Quyết định switch hiện tại có phải cores21 hay không thông qua isCore.
- Nếu là cores21 thì gọi setupCore, ngược lại gọi setupFlood.

setupFlood: Cài một flow rule flood tất cả traffic

setupCore

Setup cho cores21:

- Cài firewall rule drop với priority cao:
 - drop mọi IP từ hnotrust tới serv1
 - drop ICMP từ hnotrust tới các host nội bộ
- Cài rule đẩy ARP lên controller:
 - match dl_type ARP, output controller
 - để controller có thể học ARP và trả lời ARP gateway
- Cài rule đẩy IP lên controller:
 - match dl_type IPv4, output controller
 - để controller quyết định route và cài flow động

Cuối cùng có một rule flood priority thấp làm mặc định cho các gói không khớp rule trên.

sendToController(self, ethType, priority)

- Match theo ethType (ARP hoặc IPv4)
- Action output lên OFPP_CONTROLLER
- Tạo flow rule chuyển packet lên controller.

installDrop(self, src, dst, proto)

Tạo rule drop firewall:

- Match IPv4 theo nw_src và nw_dst.
- Nếu proto có giá trị, thêm match nw_proto để chặn riêng ICMP.
- Gửi flow_mod không có action, nghĩa là drop.

addFlow(self, match, actions)

Cài flow động cho routing:

- Tạo flow_mod priority 50, idle_timeout 30 để rule tự hết hạn khi không dùng.
- Append các action rewrite MAC và output port.
- Dùng để tăng tốc các packet sau, không phải đưa lên controller nữa.

getPortForIp(self, ip)

Hàm quyết định hướng đi dựa trên subnet của IP đích.

- Dựa trên prefix:
 - 10.0.1.x → port 1
 - 10.0.2.x → port 2
 - 10.0.3.x → port 3
 - 10.0.4.x → port 4
 - 172.16.10.x → port 5
- Trả về None nếu không thuộc subnet nào.

sendPacket(self, data, port)

Hàm gửi packet_out ra một cổng:

- Tạo ofp_packet_out, gắn raw bytes vào msg.data
- Output ra port chỉ định

buildArp(self, opcode, hwsrc, hwdst, protosrc, protodst)

Hàm để tạo ARP packet chuẩn:

- Điền đầy đủ các trường cơ bản của ARP:
 - hwtype ethernet, prototype IP
 - hwlen 6, protolen 4
 - opcode request hoặc reply
 - địa chỉ MAC/IP nguồn và MAC/IP đích
- Trả về arp packet hoàn chỉnh

sendArpRequest(self, targetIp, port)

Hàm gửi ARP request để hỏi MAC của host đích khi router chưa biết.

- Dùng buildArp tạo ARP request:
 - nguồn là gateway IP và gateway MAC của port đó
 - đích là targetIp
- Bọc vào ethernet frame broadcast
- Gửi ra đúng port để ARP nằm trong đúng subnet

sendArpReply(self, pkt, outPort, replyMac, replyIp)

Hàm trả lời ARP request hỏi gateway:

- Lấy thông tin từ ARP request gốc:
 - MAC/IP nguồn của host hỏi gateway
- Tạo ARP reply:
 - hwsrc là MAC gateway
 - protosrc là IP gateway
 - hwdst là MAC của host hỏi
 - protodst là IP của host hỏi
- Gửi về đúng cổng inPort nơi request đi vào

isBlocked(self, ipPkt)

Hàm kiểm tra chính sách firewall theo nội dung gói IP:

- Nếu src là hnotrust và dst là serv1 thì chặn tất cả.
- Nếu src là hnotrust và protocol ICMP thì chặn nếu dst là host nội bộ.

forwardIp(self, packetData, ipPkt)

Hàm xử lý chuyển tiếp IP packet:

- Lấy dstIp và xác định outPort bằng getPortForIp.
- Nếu chưa có thông tin MAC của dstIp trong arpCache:
 - đưa packetData vào pendingPackets[dstIp]
 - gửi ARP request ra outPort
 - dừng lại để chờ ARP reply
- Nếu đã biết MAC đích:
 - dstMac lấy từ arpCache
 - srcMac là MAC gateway của outPort
 - tạo match theo nw_src và nw_dst để cài flow động
 - actions gồm:
 - set MAC nguồn thành srcMac
 - set MAC đích thành dstMac
 - output ra outPort
 - cài flow bằng addFlow để tăng tốc các packet sau
 - gửi packet hiện tại bằng packet_out

_handle_PacketIn(self, event)

Logic chính:

- Nếu packet không parse được thì bỏ qua.
- Nếu không phải cores21 thì bỏ qua vì switch khác flood.

Nhánh ARP:

- Học thông tin sender:
 - arpCache[IP nguồn] = (MAC nguồn, inPort)
- Nếu là ARP request:
 - nếu targetIp trùng một trong các gatewayIps thì trả lời ARP reply bằng sendArpReply
- Nếu là ARP reply:
 - kiểm tra pendingPackets cho IP đó
 - nếu có, lấy ra từng packet đang chờ, parse lại ethernet và forwardIp tiếp

Nhánh IPv4:

- Học src IP và MAC vào arpCache.
- Nếu isBlocked trả True thì drop.
- Nếu không bị chặn thì gọi forwardIp để route.

4.2. Kết quả (Deliverables)

Ảnh chạy controller ở port 6653(do ở port mặc định thì báo lỗi)

```
root@huli-VMware20-1:/home/huli/pox# ./pox.py openflow.of_01 --port=6653 misc.part4c
controller
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:Support for Python 3 is experimental.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-15 2] connected
INFO:openflow.of_01:[00-00-00-00-00-01 3] connected
INFO:openflow.of_01:[00-00-00-00-00-03 4] connected
INFO:openflow.of_01:[00-00-00-00-00-02 5] connected
INFO:openflow.of_01:[00-00-00-00-00-1f 6] connected
```

Ảnh chạy pingall thành công như yêu cầu đề bài, cùng với đó là các flow hiện tại của mạng

```

root@huli-VMware20-1:/home/huli# sudo python3 part4.py
mininet> pingall
*** Ping: testing ping reachability
h10 -> h20 h30 X serv1
h20 -> h10 h30 X serv1
h30 -> h10 h20 X serv1
hnotrust1 -> X X X X
serv1 -> h10 h20 h30 X
*** Results: 40% dropped (12/20 received)
mininet> dpctl dump-flows
*** cores21 -----
  cookie=0x0, duration=155.515s, table=0, n_packets=2, n_bytes=196, priority=300,ip,n
w_src=172.16.10.100,nw_dst=10.0.4.10 actions=drop
  cookie=0x0, duration=155.474s, table=0, n_packets=2, n_bytes=196, priority=300,icmp
,nw_src=172.16.10.100,nw_dst=10.0.1.10 actions=drop
  cookie=0x0, duration=155.474s, table=0, n_packets=2, n_bytes=196, priority=300,icmp
,nw_src=172.16.10.100,nw_dst=10.0.2.20 actions=drop
  cookie=0x0, duration=155.474s, table=0, n_packets=2, n_bytes=196, priority=300,icmp
,nw_src=172.16.10.100,nw_dst=10.0.3.30 actions=drop
  cookie=0x0, duration=155.474s, table=0, n_packets=0, n_bytes=0, priority=300,icmp,n
w_src=172.16.10.100,nw_dst=10.0.4.10 actions=drop
  cookie=0x0, duration=155.474s, table=0, n_packets=15, n_bytes=630, priority=200,arp
actions=CONTROLLER:65535
  cookie=0x0, duration=155.474s, table=0, n_packets=22, n_bytes=2156, priority=10,ip
actions=CONTROLLER:65535
  cookie=0x0, duration=155.474s, table=0, n_packets=146, n_bytes=15700, priority=1 ac
tions=FLOOD
*** dcs31 -----
  cookie=0x0, duration=155.521s, table=0, n_packets=165, n_bytes=17095, actions=FLOOD
*** s1 -----
  cookie=0x0, duration=155.534s, table=0, n_packets=163, n_bytes=17166, actions=FLOOD
*** s2 -----
  cookie=0x0, duration=155.536s, table=0, n_packets=162, n_bytes=16829, actions=FLOOD
*** s3 -----
  cookie=0x0, duration=155.543s, table=0, n_packets=165, n_bytes=17222, actions=FLOOD
mininet> hnotrust1 iperf -c h10

```

Ảnh hnotrust1 có thể gửi IP traffic tới h10

```

"Node: h10"
root@huli-VMware20-1:/home/huli# iperf-s
iperf-s: command not found
root@huli-VMware20-1:/home/huli# iperf -s
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)

[ 1] local 10.0.1.10 port 5001 connected with 172.16.10.100 port 32896
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0006 sec 96.0 GBytes 82.5 Gbits/sec

"Node: hnotrust1"
root@huli-VMware20-1:/home/huli# iperf -c h10
h10: Host name lookup failure
root@huli-VMware20-1:/home/huli# iperf -c 10.0.1.10
Client connecting to 10.0.1.10, TCP port 5001
TCP window size: 85.3 KByte (default)

[ 1] local 172.16.10.100 port 32896 connected with 10.0.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0069 sec 96.0 GBytes 82.4 Gbits/sec
root@huli-VMware20-1:/home/huli#

```


Do bị firewall chặn nên hnotrust1 không thể gửi IP traffic tới serv1 (terminal hang)

```
mininet> hnotrust1 iperf -c h10
-----
Client connecting to 10.0.1.10, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] tcp connect to 10.0.1.10 port 5001 failed (Connection refused) on 2025-12-13 1
4:08:41 (+07)
mininet> hnotrust1 iperf -c serv1
-----
Client connecting to 10.0.4.10, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
```

V. TÀI LIỆU THAM KHẢO

- <https://en.wikipedia.org/wiki/EtherType>
- https://en.wikipedia.org/wiki/List_of_IP_protocol_numbers
- https://github.com/mininet/openflow-tutorial/wiki/Create-a-Learning-Switch#Controller_Choice_POX_Python
- <https://noxrepo.github.io/pox-doc/html/#working-with-packets-pox-lib-packet>
- <https://github.com/noxrepo/pox/blob/gar-experimental/pox/lib/packet/arp.py>
- <https://courses.cs.washington.edu/courses/cse461/23wi/projects/project2/>
- ChatGPT, Gemini